S4 explained — tools

A full walkthrough of what was built, why, and what to say when asked.

Throwaway file. Move it into `docs/` if you want to keep it, or delete it.

---

1. What a tool is

Until now the tutor knew only what was in the prompt. A tool lets it ask for more.

A student asks *"what did I study last week?"*. The model cannot know. It has never seen your database. So it has two options: make something up, or ask.

A tool is how it asks.

Here is a real turn from the test suite, step by step:

```
1. We send:   the prompt, plus a list of tools the model may use
2. It replies: "call evaluate_expression with {expression: '3 * 4 + 2'}"
               (no answer text — it is not ready to answer yet)
3. We:         look the name up, check the arguments, run it, get 14
4. We send:    the same prompt + "you asked for this, here is 14"
5. It replies: "It comes to 14."   <- now it is an answer
6. We:         run it through the S3 checks and save it
```

Two calls to Gemini, one tool run, and three database rows: two `turn_attempts` and one `tool_calls`.

The key idea: we never let the model run anything. It *asks*. We decide.

---

2. What changed in the code

The turn loop already existed from S3. It looked like this:

```
call the model → check the answer → repair if bad → save
```

Now it looks like this:

```
call the model → did it ask for tools?
                    yes → run them, add results, loop again
                    no  → check the answer → repair if bad → save
```

That is the whole change. One `if` in the middle of a loop you already had.

A tool round does not use up a repair. Nothing was *wrong* with a response that asks for a tool — it just was not an answer yet. Counting it as a repair would mean a turn that used two tools had no repairs left for a genuinely bad answer.

---

3. The five protections

The brief says: *"Tool use must be controlled; the model should not call arbitrary code or call tools indefinitely."* That is one sentence and it is the whole grading criterion for this slice. Five separate protections answer it.

Learn these five. They are the most likely thing you will be asked to walk through.

Protection 1 — a fixed list

`app/tools/registry.py`

```python
_TOOLS: dict[str, Tool] = _build()   # filled once, at import

def resolve(name: str) -> Tool | None:
    return _TOOLS.get(name)
```

That is it. A dictionary lookup. If the model asks for `os.system`, the lookup misses and returns `None`.

Say this carefully on the call, because the weak version sounds the same.

- Weak: *"we check the tool name against a list."* A check can be incomplete or forgotten.
- Strong: *"there is no mechanism by which an unlisted name could execute."*

There is no `getattr`, no `importlib`, no `eval`, no naming convention that turns a string into a function. Adding a tool means editing a file, which means it goes through review.

Protection 2 — arguments are checked before anything runs

Every tool declares its arguments as a Pydantic model:

```python
class StudyHistoryArgs(BaseModel):
    days_back: int = Field(default=30, ge=1, le=366)
```

Two things come from that one class:

1. The **declaration** we send to Gemini (generated by `model_json_schema()`)
2. The **validation** when the arguments come back

So what we told the model and what we enforce are the same object. They cannot drift apart. If the model sends `days_back: "last tuesday"`, validation fails and `run()` is never called.

The error message goes back to the model, so it can fix itself:

```
days_back: Input should be a valid integer
```

Protection 3 — identity never comes from the model

This is the security one, and it is the neatest.

Look at what a tool receives:

```python
@dataclass(frozen=True)
class ToolContext:
    db: AsyncSession
    student: Student      # <- from the X-Student-Id header, before the model ran
    room: Room
```

And look at what it does **not** receive: any student id in its arguments.

The model cannot ask for another student's data because there is nowhere to put the request.** The parameter does not exist.**

Compare that with the weaker version: accept `student_id` as an argument, then check it matches the caller. That works, until someone adds a sixth tool and forgets the check. A missing parameter cannot be forgotten.

There is a test that enforces this for every tool, present and future:

```python
def test_no_tool_accepts_a_student_id() -> None:
    for spec in registry.specs():
        fields = set(spec.parameters.get("properties", {}))
        assert not {"student_id", "user_id", "room_id"} & fields
```

Protection 4 — hard limits

Four budgets, plus a clock over all of them.

Budget	What it counts	Default
`max_agent_iterations`	rounds of ask → run → ask	5
`max_tool_calls_per_turn`	individual tool runs	6
`tool_timeout_seconds`	one tool	8
`turn_deadline_seconds`	the whole turn	45

The interesting design choice: running out of budget does not fail the turn.

When the budgets run out, we simply stop *offering* the tools:

```
offer_tools = (
    purpose is AttemptPurpose.TUTOR and rounds_left > 0 and tool_calls_left > 0
)
```

The model then has to answer with what it already gathered. A student waiting gets an answer instead of an error. The loop still terminates — it just narrows.

That is a better answer than "we fail the turn", and it is worth saying out loud.

Protection 5 — the same call twice

If the model asks for the identical tool with identical arguments, we return the saved result plus a note:

```
"You already called this with these arguments. This is the same result.
 Do not call it again — answer the student now."
```

Limits end a loop *eventually*. This ends it *now*.

One detail worth knowing: the cache key sorts the argument keys.

```
json.dumps(invocation.arguments, sort_keys=True)
```

Without `sort_keys`, `{"a":1,"b":2}` and `{"b":2,"a":1}` would be different keys, the cache would miss, and you would pay for the repeat.

---

4. The calculator, in depth

`app/tools/calculator.py` is the file most likely to be attacked on the call. It takes a string written by a language model and evaluates it. That is the most dangerous shape a function can have.

Why not `eval`

Because this works:

```
eval("__import__('os').system('rm -rf /')")
```

And so does this, which needs no imports at all:

```
eval("(1).__class__.__bases__[0].__subclasses__()")
```

Read that second one. From the number `1` you reach its class, then that class's parent (`object`), then **every class Python has loaded** — which includes file handles and subprocess wrappers. From an integer to the file system in three steps.

You cannot fix this by filtering the input string. The attacker chooses the string, and Python's object graph is fully connected. There is always another route. The fix is to never call `eval`.

What we do instead

Parse the expression into a tree, then check **every node** against an allow-list:

```
_ALLOWED_NODES = (
    ast.Expression, ast.Constant, ast.BinOp,
    ast.UnaryOp, ast.Call, ast.Name, ast.Load,
)
```

Seven node types. Anything else is rejected before any of it runs.

`ast.Attribute` **is not on the list.** That means the `.` character cannot be written at all. Every escape above starts with a `.`, so none of them can even be expressed.

That is why the argument is strong. It is not *"we thought of the bad inputs"* — it is *"only these seven things can run, and none of them can reach an attribute, a name, an import, or a loop."*

The one thing an allow-list does not catch

```
9 ** 9 ** 9
```

Every node in that is allowed. It is just numbers and an operator. But the result has about 370 million digits, and Python will take every core you have trying to compute it.

An allow-list cannot see this. **Size limits** catch it instead:

```
MAX_EXPONENT = 1000
MAX_BASE_FOR_POWER = 10**15
MAX_FACTORIAL_INPUT = 1000
MAX_EXPRESSION_LENGTH = 500
```

Note also that a computed exponent is refused outright:

```
9 ** (3 * 5000)   ->  "The exponent must be a plain number, not another calculation."
```

You cannot know how big that is without evaluating it, and evaluating it is the thing being avoided.

Also worth knowing: the 8-second tool timeout does *not* protect against this. `asyncio.wait_for` cannot interrupt synchronous CPU work. The size limits are the only defence here. That is a good thing to say if asked — it shows you know what the timeout does and does not do.

The tests

`tests/test_calculator.py` runs eleven real attacks, including both subclass walks. The test asserts only that each was **rejected**, not which rule caught it — several trip more than one, and which fires first depends on tree walk order. Pinning that would make the test brittle without making the guarantee stronger.

---

5. Study history

Two doors, one room

The brief lists study history twice: once as an acceptance item (an endpoint) and once as its example tool. Those are the same question asked two ways.

So the query lives in **one** place, `app/services/study_history.py`, and both call it:

```
GET /students/me/study-history   ─┐
                                  ├──> get_study_history()
query_study_history tool         ─┘
```

Written twice, they would drift, and then "what did I study" would depend on who was asking.

What counts as studying

Only turns that **succeeded** and were **in scope**.

- A failed turn taught nothing.
- An off-topic turn is not revision of the room's subject.

This is the reader that `in_scope` was stored for back in S3. Without it, a student who chatted about the World Cup in an algebra room would appear to have revised algebra.

The concept counting query

This is the one piece of unusual SQL. `turns.concepts` is a JSON list:

```
turn 1: ["linear equations", "inverse operations"]
turn 2: ["linear equations"]
```

To count them, the list must be expanded into one row per tag:

```
FROM turns
JOIN rooms ON rooms.id = turns.room_id
JOIN LATERAL jsonb_array_elements_text(turns.concepts) AS tag(concept) ON true
GROUP BY tag.concept
```

What `LATERAL` means: normally a joined table cannot see the current row. It is computed once. `LATERAL` means "run this again for every row, and let it look at that row" — which is required here, because the function reads `turns.concepts` from the row being processed.

Why the tags are counted and not stored: storing running totals would be a second copy of a fact already recorded, and second copies drift. Delete a turn and the total is silently wrong with nothing to notice it.

Why the tool takes `days_back` and the endpoint takes dates

The model does not reliably know today's date. Ask it for `from_date` and it will confidently give you the wrong one — and a wrong date range returns an empty result that reads as *"you studied nothing"*, which is worse than an error.

`days_back=7` needs no knowledge of the calendar. The endpoint keeps full dates, for callers who do know what day it is.

---

6. The constraint that shaped everything

Before writing any code I ran three small real API calls. The result:

```
tools only           -> works
response_schema only  -> works
tools + response_schema -> 400 INVALID_ARGUMENT
  "Function calling with a response mime type: 'application/json' is unsupported"
```

Gemini will not let you use tools and structured output at the same time.

This matters, because in S3 you added `response_schema` to fix a real bug (the model answering in prose, restating it as JSON, and running out of budget mid-way).

So the design has to choose, per call:

Call	Tools	response_schema
Gathering (may use tools)	yes	**no**
Budgets spent, must answer	no	yes
Repair	no	yes

`LLMRequest.__post_init__` refuses the combination outright, so it fails in our code with a clear message rather than at the API with one nobody reads.

The point worth making on the call

> "Constrained decoding is unavailable exactly while the model can call tools. So during a tool-using turn, the parse → schema → content pipeline is the *only* thing between model output and the student. `response_schema` did not make that pipeline redundant — it just narrowed where it is the sole defence."

That justifies S3 existing, and it is backed by an error message from the provider rather than an opinion.

---

7. Three real bugs found while building

Good material — these are honest and specific, and every one was caught by something rather than noticed by luck.

The CHECK constraint caught a half-written row

`tool_calls` has a rule: an `ok` row has a result, a failed row has an error.

The first time `query_study_history` ran, the insert failed. The reason: the tool runs a `SELECT`, and SQLAlchemy **autoflushes** pending objects before a query. The half-built row — status `ok`, no result yet — got written mid-call and broke the rule.

The fix: build the row, run the tool, *then* add it.

Note this is the opposite of `turn_attempts`, which **is** written before its call. The difference is what a crash costs: an attempt holds the prompt, which is unrecoverable; everything about a tool call is already known from the attempt that asked for it.

SQL that compiled and then failed

The concept query first used `.column_valued()`, which produced a cartesian-product warning on every call. Switching to an explicit `LATERAL` printed clean SQL:

```
JOIN LATERAL jsonb_array_elements_text(turns.concepts) AS anon_1 ON true
```

Then it failed at the database. Look closely — there is no `(concept)` after `anon_1`, so the column had the function's own name and `anon_1.concept` did not exist. `render_derived()` restores the alias.

Lesson: printing SQL is not running SQL. It was fixed by executing it, not by reading it.

An index that duplicated a constraint

`tool_calls` originally had both:

```
UNIQUE (attempt_id, call_no)
INDEX  (attempt_id, call_no)
```

Postgres builds a btree for a UNIQUE constraint. The second index was identical — a write on every insert, for nothing. Removed.

---

8. Files

New:

```
app/tools/base.py            what a tool is; ToolContext; ToolStatus
app/tools/registry.py        the fixed list, and argument validation
app/tools/calculator.py      evaluate_expression, AST allow-list
```

```
app/tools/study_history.py   query_study_history
app/services/study_history.py  the shared query
app/models/tool_call.py      one row per tool run
app/schemas/study_history.py
prompts/tutor_system/v3.md   adds guidance on when to reach for a tool
scripts/seed_demo_data.py    six weeks of history to query
migrations/versions/20260813_0900_tool_calls.py
tests/test_tools.py          24 tests
tests/test_calculator.py     38 tests
tests/test_study_history.py  13 tests
```

Changed:

```
app/llm/base.py         ToolSpec, ToolInvocation, ToolResult; tools on the request
app/llm/gemini.py       declares tools, builds multi-turn contents, reads function calls
app/services/turn.py    the tool loop and _run_tool
app/schemas/turn.py     ToolCallRead nested under AttemptRead
app/api/routes/students.py  the study-history endpoint
app/config.py           the budgets
tests/fakes.py          FakeLLM can script tool requests
```

179 tests pass. Ruff clean. Migration round-trips with no drift.

---

9. Questions you should expect

"How do you stop the model calling arbitrary code?" Five protections. Lead with the registry: a dict lookup, no dynamic dispatch anywhere. Then the calculator's AST allow-list — `ast.Attribute` is not permitted, so `.` cannot be written, so the subclass escape cannot even be typed.

"What stops it looping forever?" Four budgets and a deadline. And the good detail: when they run out we stop

- offering* the tools, so the model must answer with what it has. The turn narrows instead of failing. There is a runaway-loop test.

"Could a tool read another student's data?" No — there is no argument for it. Show `ToolContext`, then the test that asserts no tool has a `student_id` field. The parameter does not exist to be filled in.

"Why only two tools?" The four-part test in `docs/PLAN.md` §8, plus the table of everything it rejected and why. Lead with the test, not the list.

"Why didn't you use structured output with tools?" Because Gemini returns 400. Quote the message. Then explain what it implies for where the reliability pipeline matters.

"Why is the calculator not just sympy?" sympy would evaluate the expression, but `sympify` on raw input has had sandbox escapes. The AST allow-list is auditable in one screen — you can read the seven node types and the twenty functions and know what can run. sympy is in the plan for S6, where symbolic work genuinely needs it.

"What's the weakest part of this?" Answer honestly: the 8-second tool timeout cannot interrupt synchronous CPU work, so the calculator's protection is really its size limits, not the timeout. And a tool timeout fails the whole turn, because cancelling a database query mid-flight leaves the session in a state not worth writing through. Both are deliberate and documented — saying so is stronger than pretending neither exists.